

CA3 team plan: what changes and who does what

Team AHD · AI-Powered Multi-Agent IoT Monitoring and Predictive Maintenance System · submission deadline **Friday 9 October 2026**

Our project is approved (strength 7/10). The CA3 guidelines grade things we had not planned for: a live public link, proof that our agents really reason and coordinate, and each member's own GitHub activity. This document is our plan for the next seven days. Please read it fully, including the GitHub rules, before your first commit.

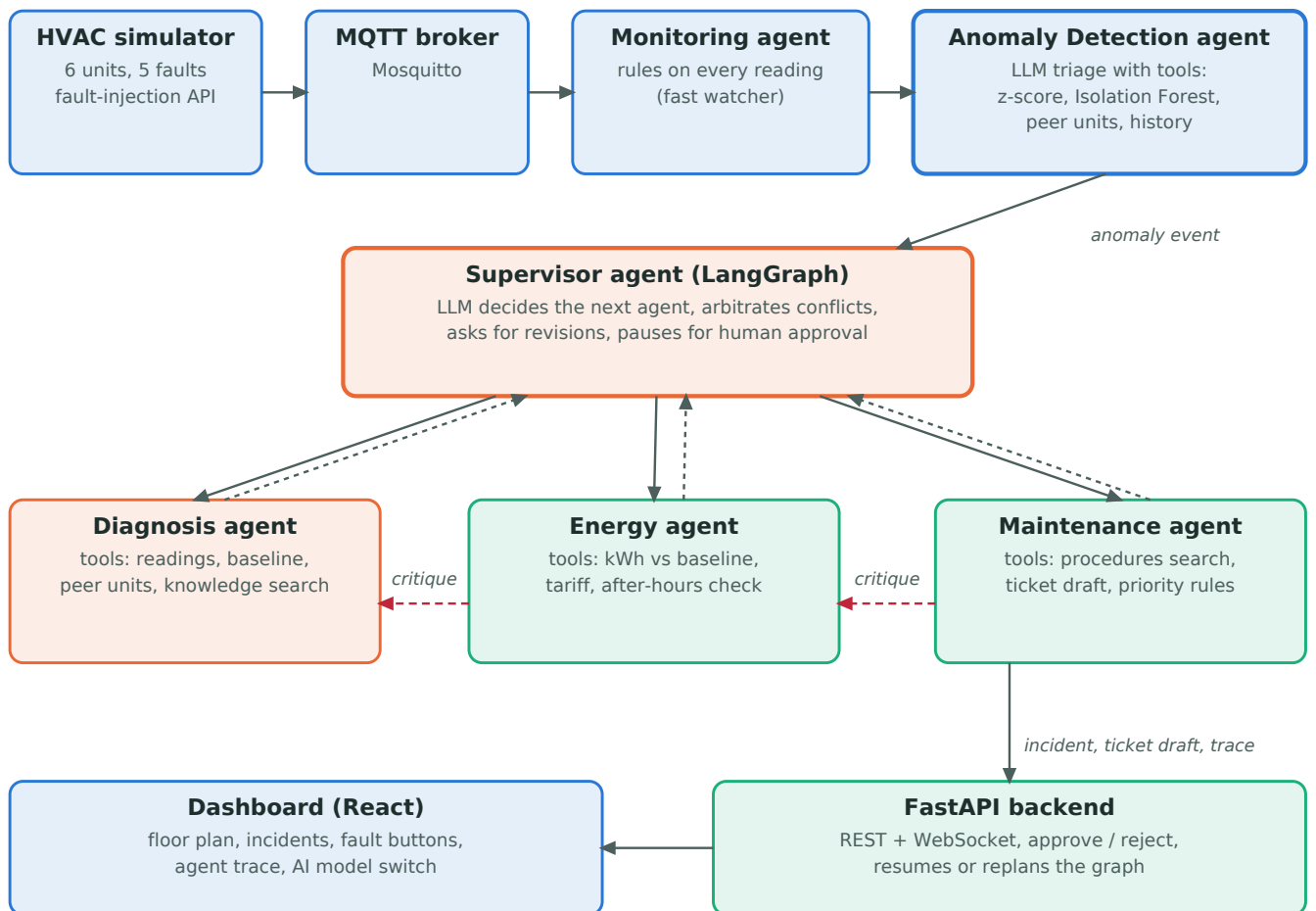
1. How we will be marked (45 marks)

Category	Marks	What it means for us
A. Multi-agent architecture & agentic design	8	At least 4 agents that plan, use tools and adapt. Evidence comes mainly from a real execution trace. A fixed chain of steps scores low.
B. Design	8	Architecture diagram with every agent and connection, roles, orchestration pattern, input/output schemas with real samples.
C. Live demo	6	A public URL that works on demo day and stays up until results are published. Graceful handling of bad input and failing tools.
D. Presentation	8	Explain why we need several agents, justify our choices, and each member answers questions about their own part .
E. Documentation	5	Synopsis, design document and the deployed system must all describe the same thing.
F. Code & engineering	5	Clean repo, README, .gitignore, no secrets, real tests, meaningful commits over several days.
G. Individual contribution	5	Per person , from your own GitHub account: commits, number of different days, pull requests opened and reviewed.

What the reviewer asked us to fix: “Doesn't describe any replanning or agent-to-agent negotiation when findings conflict (e.g. Diagnosis vs. Energy Optimization disagreeing on root cause); fairly light on the specific tools/APIs each agent would use.” Section 3 below answers exactly this.

2. The system

Six simulated air handling units send sensor data over MQTT. The Monitoring agent watches every reading; the Anomaly Detection agent decides whether something is really wrong. The Supervisor then decides, case by case, which specialist agents to involve, resolves disagreements between them and asks a person to approve the final recommendation.



■ **Ali**: data, detection, dashboard, deployment

■ **Ahmed**: supervisor, diagnosis, LLM layer, trace

■ **Abdullah**: energy, maintenance, knowledge, API

Orchestration pattern: **supervisor (orchestrator-worker) with a critique loop**. Specialists can critique each other's results; the Supervisor arbitrates and may send work back for revision (at most two rounds) before escalating to the human.

3. What makes it agentic (we must show all five in the trace)

Behaviour	What happens	Owner
Dynamic routing	The Supervisor's LLM chooses the next agent from the evidence. After-hours waste goes to Energy first; a frozen sensor goes from Diagnosis straight to Maintenance (no energy cost); a compressor failure goes through all three.	Ahmed
Conflict resolution	If Energy's numbers contradict the diagnosis (e.g. Diagnosis says compressor failure but power went up), Energy sends a critique. The Supervisor returns the case to Diagnosis with that evidence. If they still disagree after two rounds, both views go to the human.	Ahmed + Abdulelah
Asking for more evidence	Maintenance refuses to draft a ticket when diagnosis confidence is below 0.6 and asks for more data. Diagnosis then calls extra tools (longer history, peer units) and tries again.	Abdulelah + Ahmed
Replanning after rejection	When the operator rejects a recommendation, the reason becomes new evidence and the Supervisor replans instead of stopping.	Ahmed
Triage before escalation	The Anomaly Detection agent decides with tools whether to escalate, keep watching, or dismiss (e.g. all units deviating together means weather).	Ali
Failure handling	If a tool or the LLM fails (timeout, bad JSON, no internet), the agent retries, switches provider (OpenAI to Ollama to rules) and writes this into the trace instead of crashing.	All

Agents, tools and model choice

Agent	Tools it can call	Model	Owner
Monitoring	threshold and rate-of-change rules, offline watcher	none (runs on every reading, must be instant)	Ali
Anomaly Detection	zscore_check, isolation_forest_score, compare_peer_units, get_history	small, fast model	Ali
Supervisor	route_to(agent), request_revision, escalate_to_human	OpenAI (most reliable routing)	Ahmed
Diagnosis	get_recent_readings, compare_with_baseline, get_peer_units, search_knowledge, get_incident_history	OpenAI; Ollama as offline backup	Ahmed
Energy	compute_extra_kwh, get_tariff, check_after_hours, critique_diagnosis	calculations in code, LLM only explains	Abdulelah
Maintenance	search_procedures, draft_ticket, set_priority, request_more_evidence	small model is enough	Abdulelah

All agents call one function, `llm.complete(prompt, schema, agent=...)`, so the provider can be switched live from the dashboard (OpenAI, Ollama, rules only) and chosen per agent.

4. Who does what

■ Ali: data, detection, dashboard, deployment

- Already built and tested: simulator with 5 fault types, labelled datasets, Monitoring rules, z-score and Isolation Forest detection (50/50 test faults found, median 8 min), MQTT ingestion, React dashboard. Will add these to the repo in reviewable pull requests.
- Turn the Anomaly Detection agent into an LLM agent that uses the detectors as tools.
- Fault-injection buttons on the dashboard and a simulation-control API, so evaluators can trigger faults themselves.
- Input validation (bad sensor messages get a clear error, not a crash).
- Deploy everything to a cloud server and keep the live URL up.
- Assemble the synopsis, design document and input/output definitions; record the demo video.

■ Ahmed: orchestration, diagnosis, LLM layer, trace

- LLM layer: OpenAI, Ollama and rules-only behind one function, with retry and fallback.
- Shared data models in backend/app/agents/schemas.py (agree them with Abdulelah on day 1).
- LangGraph graph: Supervisor with LLM routing, revision loop (max 2), human approval pause, replanning after rejection.
- Diagnosis agent and its tools.
- Execution trace: every message, tool call and decision saved with time, sender and receiver; downloadable per incident.
- Tests for the routing and conflict loop (using a fake LLM so they run without internet).

■ Abdulelah: specialist agents, knowledge, API

- Knowledge base: about 10 short HVAC maintenance notes and 20 past incident records, plus the search tool.
- Energy agent (extra kWh and cost, after-hours check, critiques the diagnosis when numbers disagree).
- Maintenance agent (procedures, ticket draft, priority, asks for more evidence when confidence is low).
- FastAPI backend: incidents, approve and reject, tickets, energy summary, agent activity, AI switch, WebSocket. The example responses in contracts/api_examples/ are the exact shapes the dashboard expects.
- Diagnosis accuracy test: OpenAI vs Ollama vs rules on the 50 labelled test faults.
- Tests for the energy calculation, the knowledge search and the API.

Ahmed and Abdulelah meet at the data models in schemas.py (Diagnosis, EnergyImpact, Recommendation, Critique, IncidentState). Write them together on the first day; after that each can work alone. Everyone must be able to explain the whole system, not only their part.

5. Seven-day plan

Day	Ali	Ahmed	Abdulelah
Fri 2 Oct	Create repo, add both of you, PR 1: contracts + README	Accept invite, register GitHub username on Exam Studio, install Python	Same as Ahmed
Sat 3 Oct	PR: simulator + datasets. Choose cloud server	With Abdulelah: schemas.py. LLM layer with fallback	With Ahmed: schemas.py. Knowledge base files + search tool
Sun 4 Oct	PR: detection + ingestion. Start LLM triage agent	LangGraph graph with Supervisor routing	Energy agent
Mon 5 Oct	PR: dashboard + fault buttons + control API	Diagnosis agent + tools; revision loop	Maintenance agent; API endpoints
Tue 6 Oct	Deploy to server; integrate everything	Human approval + replanning; execution trace	API connected to the graph; WebSocket
Wed 7 Oct	Robustness: bad input, offline unit. Detection results	Tests; capture real traces of 3 scenarios	Tests; diagnosis accuracy experiment
Thu 8 Oct	Design doc, synopsis, I/O definitions; demo video	Write your design-doc sections; slides	Write your design-doc sections; slides
Fri 9 Oct	Final check of live URL, submit artifacts	Rehearse Q&A; on your part	Rehearse Q&A; on your part

Every day: commit what you did, push, and review one pull request from a teammate.

6. GitHub: setup and rules

Setup (once)

- Ali creates a **public** repository and adds Ahmed and Abdulelah as collaborators (Settings → Collaborators). Accept the invite from your email.
- Each of us registers our GitHub username on Exam Studio → Project Artifacts. Marks are matched by username.
- Make your commits count for **your** account: set the same email that is on your GitHub account.

```
git config --global user.name "Your Name"
git config --global user.email "the-email-on-your-github-account"
```

- The main branch is protected: changes go in through pull requests with one approval.

Daily routine (10 to 15 minutes)

```
git checkout main && git pull
git checkout -b feat/energy-agent # one branch per task
# ... work ...
git add -A && git commit -m "Energy agent: compute extra kWh against baseline"
git push -u origin feat/energy-agent
# open a pull request on GitHub, ask a teammate to review
```

- Commit at the end of every work session with a message that says what changed. Small commits on several days score far better than one big upload.
- Review at least one teammate's pull request per day: read it, leave a real comment, approve. Reviews are counted for the reviewer.

Never commit API keys (they live in `.env`, which git ignores), commit under someone else's account, or change commit dates. The guidelines treat commit-history manipulation and edited execution traces as academic integrity violations, and GitHub records when every commit was really pushed.

7. What we submit by 9 October

Artifact	Content	Owner
Project synopsis	Problem, solution, target user (2 to 3 paragraphs); note any scope change	Ali
Design document	Diagram, agent roles, orchestration pattern, technology choices with reasons	Ali , sections by all
Input definition	Telemetry schema + real sample (already in contracts/)	Ali
Output definition	Incident/recommendation schema + real sample	Abdulelah
GitHub URL	Public repo, all three usernames registered	All
Project URL	Live deployment, stays up until results are published	Ali
Execution trace	Real captured runs showing agent messages, critiques and replanning	Ahmed
Demo video (optional)	2 to 3 minute walkthrough, backup only	Ali
Presentation	Slides; each member presents and answers for their own part	All